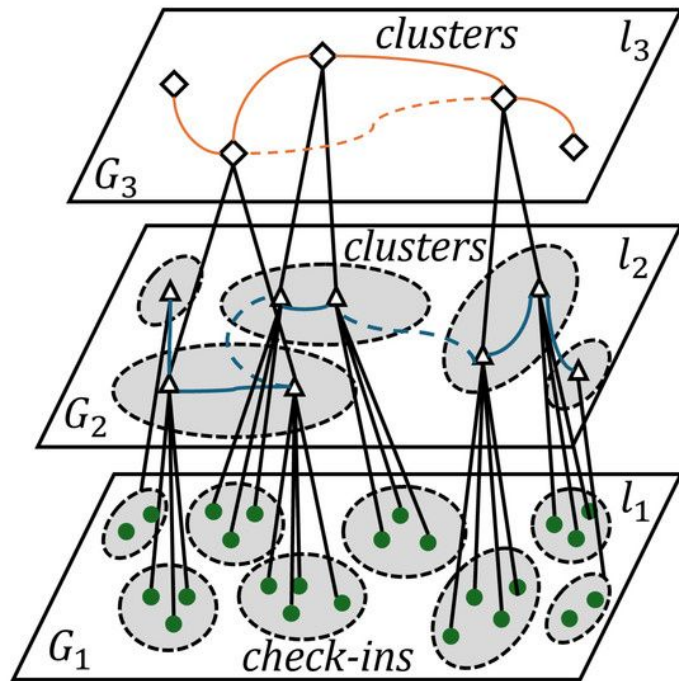


HiRAG

Retrieval-Augmented
Generation with
Hierarchical Knowledge

Paul Béglin
Ekaterina Bogush
Zahra Siddique



Plan



1

Concept de HiRAG

2

Problèmes de HiRAG

3

Améliorations implémentées

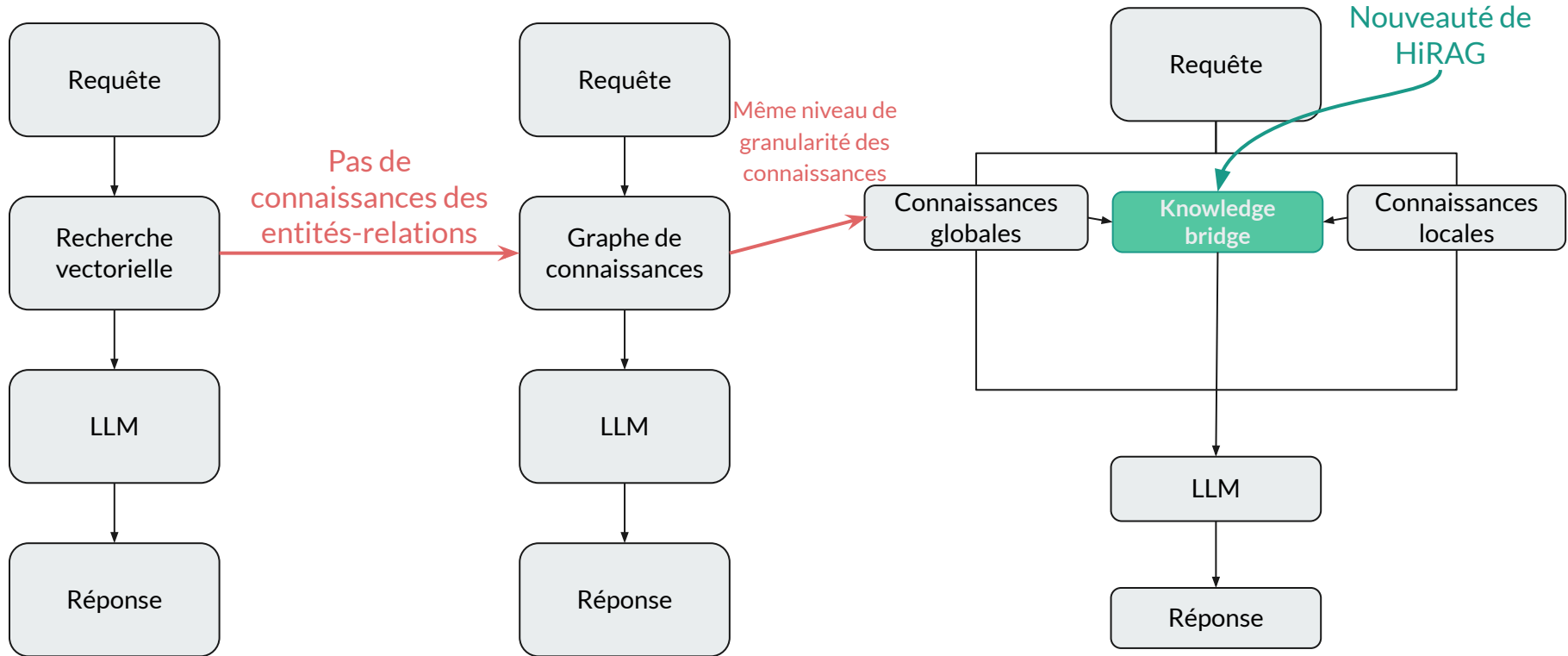
4

Évaluation

5

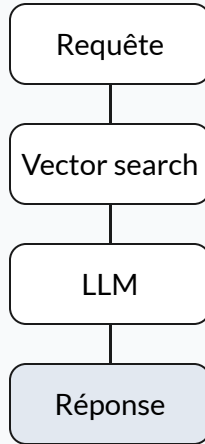
Conclusion

Concept de HiRAG. Comment améliorer le RAG?



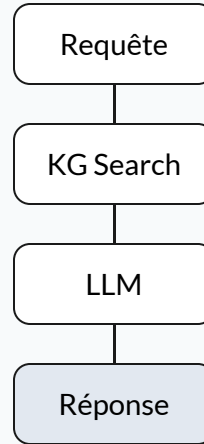
Concept de HiRAG : Comment améliorer le RAG ?

Standard RAG



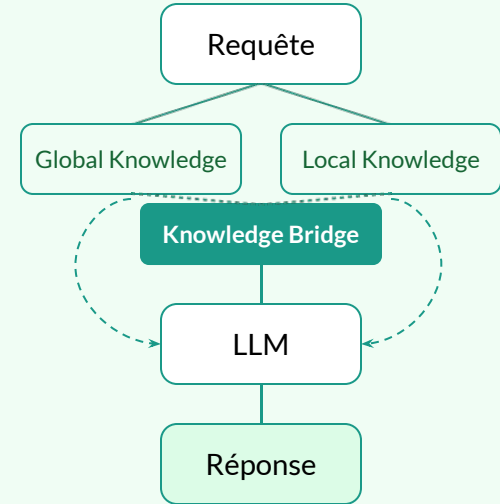
Pas de connaissances des entités-relations

Graph RAG



Même granularité des entités-relations
(difficile pour les requêtes vastes)

HiRAG (Hierarchical)



Nouveauté de HiRAG

Problèmes de HiRAG

1

Les chunks pertinents **entiers** sont passés à LLM



Non mentionné dans l'article comme faisant partie de l'implémentation

After retrieving the three-level hierarchical knowledge, i.e., local-level descriptions of the individual entities in $\hat{\mathcal{V}}$, global-level community reports of the communities in $\hat{\mathcal{P}}$, and bridge-level descriptions of the triples in $\hat{\mathcal{R}}$, we feed them as the context to the LLM to generate a comprehensive answer to the query. We also provide the detailed procedures of HiRAG with pseudocodes in Appendix C.

2

Trouver des entités locales pertinentes avec la **recherche dense** seulement, pas de **reranker**

3

Bridge entre les entités locales et les communautés globales n'est pas **query-aware**

Saturation du contexte du LLM

Solution : séparer le chunk en phrases/fenêtres (selon la punctuation and `\n`), calculer le score pour chaque phrase, choisir une fenêtre par chunk

Stratégie de chevauchement avec la requête

$$\text{overlap}(Q, W) = |Q \cap T(W)|$$

$$\text{density}(Q, W) = \frac{|Q \cap T(W)|}{\max(1, |T(W)|)}$$

$$\text{score}(W) = \left(\text{overlap}(Q, W), \text{density}(Q, W), -|W| \right)$$

$$W^* = \arg \max_W \text{score}(W)$$

Q = ensemble des tokens normalisés de la requête

$T(W)$ = fenêtre candidate de texte

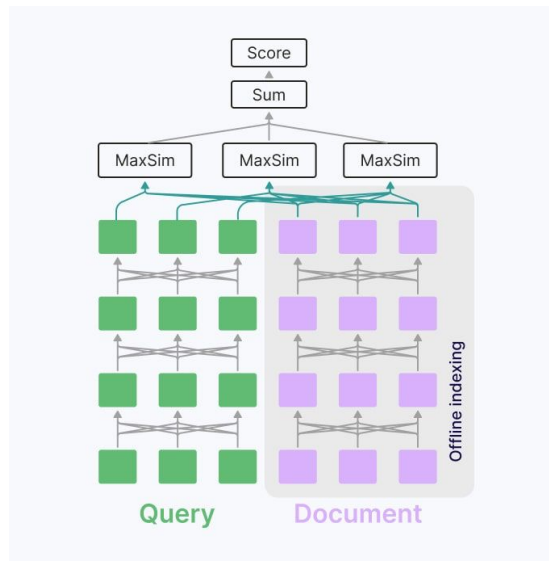
W = set of normalized tokens in window

Normalisation = mise en minuscules, tokenization, suppression des tokens d'un seul caractère, suppression des stopwords

Reranker. Mécanisme late-interaction ColBERT

Solution : recherche dense sur les entités locales (**top-100**), ensuite applique le reranker ColBERT et prendre top-20

Modèle : [answerdotai/answerai-colbert-small-v1](#)

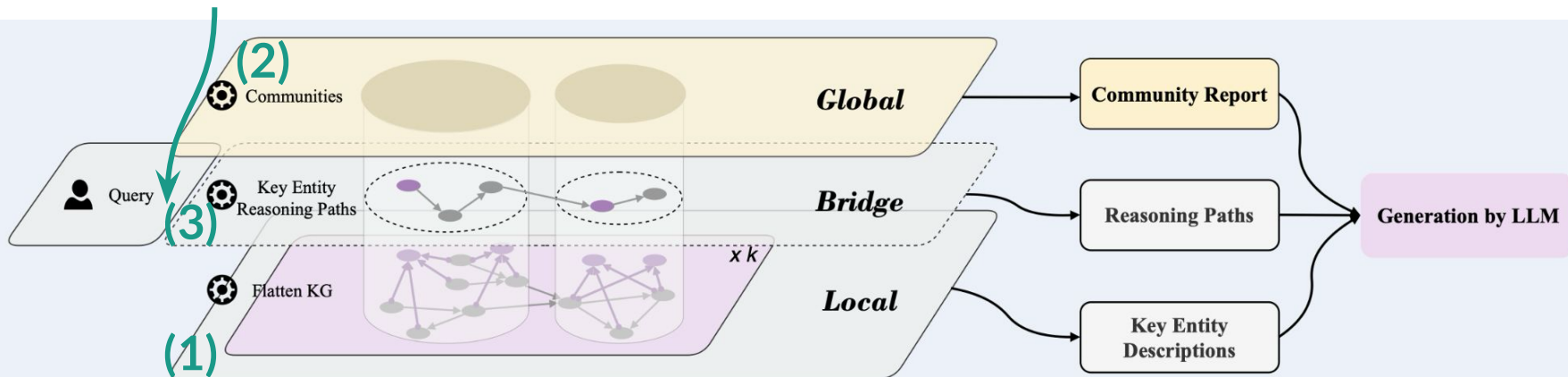


Query-aware bridge

Idée de HiRAG :

knowledge bridge = le plus court
chemin entre les entités locales,
ordonnées par leurs
communautés

Conditionner la recherche
du chemin par la requête



HiRetrieval: Retrieval with Hierarchical Knowledge

Query-aware bridge

1

Dijkstra pondéré : Parmi les chemins **complets**, lequel a le coût total minimal ?

2

Minimax : Parmi les chemins **complets**, lequel a le meilleur 'worst edge' ?

2.1

Minimax avec le budget : Parmi tous les **petits** chemins **complets**, lequel a la meilleur pire arête ?

3

Monte Carlo Tree Search : En construisant un bridge, quel est le meilleur choix garantissant une récompense long terme ?

Coût d'une arête

1 Embedding d'une arête

Calculer embedding sur

```
source: <source entity name>
<source entity description>
relation: <relation description>
target: <target entity name>
<target entity description>
```

2 Définir le coût d'une arête $e = (u, v)$ par rapport à la requête q

$$\text{similarity_score}(q, e) = \cos(\mathbf{q}, \mathbf{e})$$

$$\text{bounded_distance}(q, e) = \max(0, 1 - \text{similarity_score}(q, e))$$

minimiser $c(e | q) = \text{bounded_distance}(q, e) + 0.15 \cdot \text{hub_penalty}(e) + 0.05 \cdot \text{inverse_weight_penalty}(e)$

maximiser $u(e | q) = \max(0, 1 - c(e | q))$ utilité (utilisée en MCTS)

Pénaliser des noeuds génériques

Pénaliser les relations faibles plus que les fortes

$$\text{hub_penalty}(e) = \frac{\deg(u) + \deg(v)}{\max_{(i,j) \in E} (\deg(i) + \deg(j))}$$

$$\text{inverse_weight_penalty}(e) = \frac{1}{1 + w_e}$$

Relationship strength (définie pendant la construction du graphe)



Coût d'une arête

1 Embedding d'une arête

```
source: <source entity name>
<source entity description>
relation: <relation description>
target: <target entity name>
```

2 Coût d'une arête $e = (u, v)$

Calculé par rapport à la requête q

Objectif : Équilibrer distance, centralité et force des liens.

$$\text{similarity_score}(q, e) = \cos(\mathbf{q}, \mathbf{e}) \quad \text{bounded_distance}(q, e) = \max(0, 1 - \text{similarity_score}(q, e))$$

$$c(e | q) = \text{bounded_distance}(q, e) + 0.15 \cdot \text{hub_penalty}(e) + 0.05 \cdot \text{inverse_weight_penalty}(e) \quad \text{minimize cost}$$

$$u(e | q) = \max(0, 1 - c(e | q)) \quad \text{maximize utility}$$

utilisée en MCTS

Hub Penalty

Pénalise les noeuds trop génériques pour favoriser la spécificité des relations.

$$\text{hub_penalty}(e) = \frac{\log(d(u)) + \log(d(v))}{\log(d(u)) \cdot \log(d(v))}$$

Weight Penalty

Pénalise les relations faibles pour renforcer la cohérence.

$$\text{inverse_weight_penalty}(e) = \frac{1}{1 + u}$$

Utilité MCTS

Maximise l'intérêt informationnel local pour le raisonnement.

Dijkstra

Objective de HiRAG (par défaut) $\arg \min_P |P|$

$$P = (e_1, e_2, \dots, e_m)$$

Dijkstra pondéré $P^* = \arg \min_P \sum_{k=1}^m c(e_k \mid q)$

Limitation connue : minimise la somme des coûts, peut accepter une arête à coût conséquent si toutes les autres sont gratuites

Dijkstra pondéré

Objective de HiRAG (par défaut)

$$\arg \min_P |P| \quad P = (e_1, e_2, \dots, e_m)$$

Dijkstra pondéré

$$P^* = \arg \min_P \sum_{k=1}^m c(e_k \mid q)$$



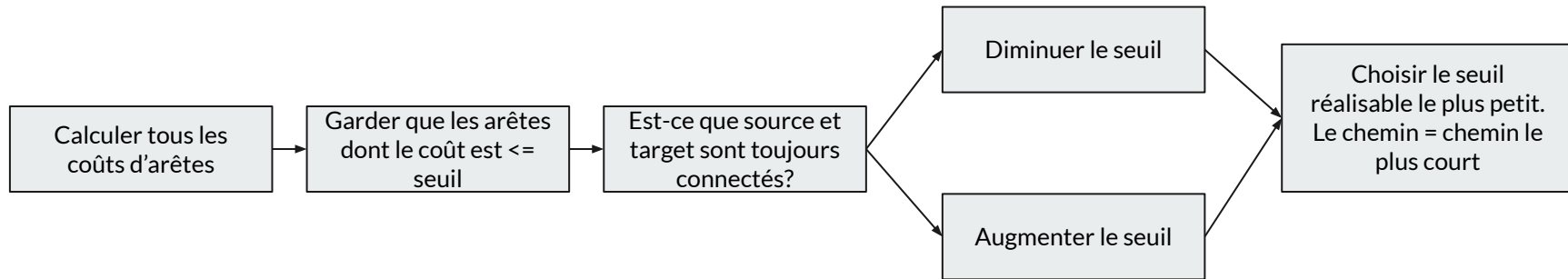
Limitation connue

Minimise la somme des coûts globale. Cela peut conduire à l'acceptation d'une arête à **coût conséquent** si la majorité des autres arêtes du chemin sont gratuites.

Minimax

$$P = (e_1, e_2, \dots, e_m)$$

Version à seuil de minimax pour **simplifier l'implémentation à budget** de minimax



Minimax classique

$$P^* = \arg \min_P \max_{e \in P} c(e \mid q)$$

Minimax avec le budget

Budget local

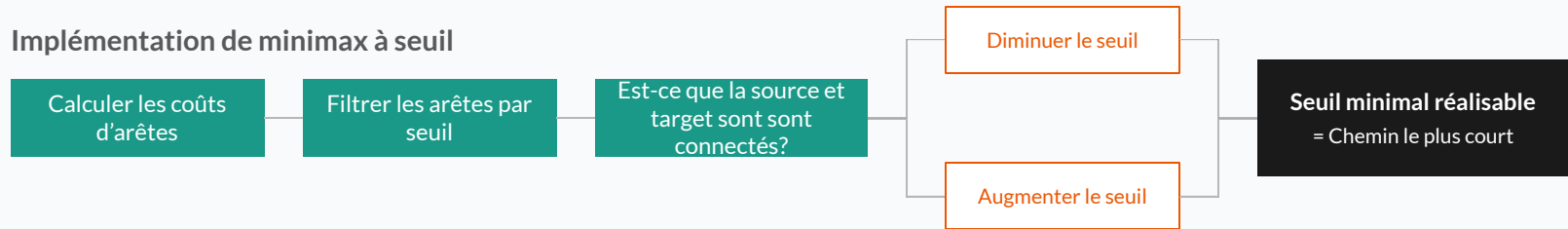
$$P^* = \min_P \max_{e \in P} c(e \mid q) \quad \text{s.t.} \quad |P| - 1 \leq 120$$

Budget global

$$\sum_i (|P_i| - 1) \leq 220 \quad (v_1 \rightarrow v_2), (v_2 \rightarrow v_3), \dots, (v_{k-1} \rightarrow v_k)$$

Algorithme Minimax

Implémentation de minimax à seuil



Version à seuil simplifiée pour l'implémentation à budget.

Minimax Classique

$$P^* = \arg \min_P \max_{e \in P} c(e \mid q)$$

Minimax avec Budget

Budget Local

$$P^* = \min_P \max_{e \in P} c(e \mid q) \quad \text{s.t.} \quad |P| - 1 \leq 120$$

Budget Global

$$\sum_i (|P_i| - 1) \leq 220 \quad (v_1 \rightarrow v_2), (v_2 \rightarrow v_3), \dots, (v_{k-1} \rightarrow v_k)$$

Monte Carlo Tree Search (MCTS)

Trouver un chemin de liaison qui utilise des arêtes pertinentes, atteint la cible, reste court et ne consomme pas trop tokens.

1

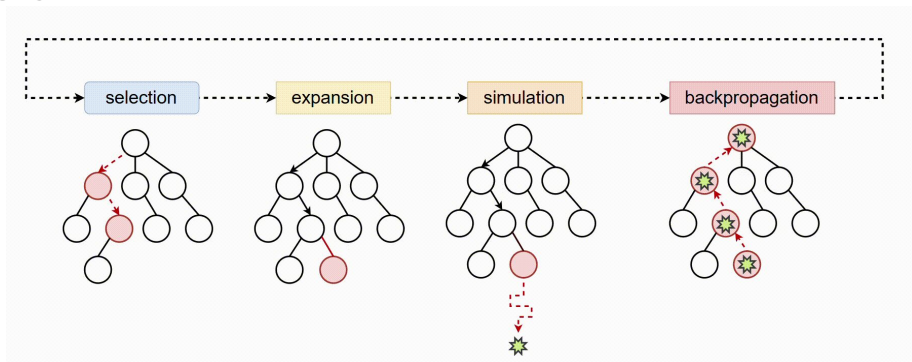
Calculer le coût des arêtes

2

Restreindre l'espace de recherche à un **sous graphe candidat** autour de la source et target.
Permet d'accélérer la convergence et limiter le branching factor

3

Appliquer MCTS



4

Si le chemin trouvé par MCTS est meilleur que Dijkstra, on arrête l'exploration (**early stopping**).

Construire le sous-graphe

- 1 Restreindre le graphe aux voisins des nœuds source et cible situés dans un rayon de r sauts

$$V_{\text{cand}} = N_r(s) \cup N_r(t) \cup \{s, t\}$$

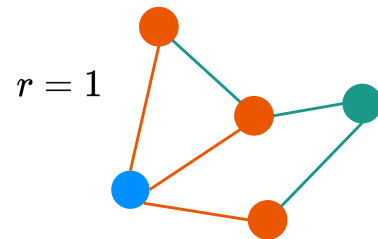
$$r \in \{2, 3, 4\}$$

↑
augmente si la source et la cible
deviennent déconnectées

- 2 **Pruning** : Pour chaque nœud v , prendre que les voisins avec la meilleure utilité et les moins génériques

$$\mathcal{N}_{\text{keep}}(v) = \text{Top-12}_{x \in \mathcal{N}(v)} (u(v, x \mid q), -\deg(x))$$

- 3 Si le nœud source et cible devient déconnectées, éviter le pruning



Appliquer MCTS

3.0

Initialiser la racine de l'arbre d'exploration

noeuds visités à l'instant t

nombre de tokens estimés

noeud courant

chemin à l'instant t

source

$$s_0 = (v_0, P_0, V_0, \tau_0) = (s, [s], \{s\}, 0)$$

3.1

Sélection : choisir les enfants avec le meilleur UCT

nombre de fois l'état s a été visité

$$UCT(s, a) = \bar{Q}(s, a) + 1.2 \sqrt{\frac{\ln N(s)}{N(s, a)}}$$

Hypothèse souvent posée en MCTS

$$s \xrightarrow{a} s'$$

$$Q(s, a) = Q(s')$$

nombre de fois l'action a a été pris l'état s

Pour le noeud-enfant non-visité (pour garantir l'exploration)

$$UCT(s, a) = +\infty$$

3.3

Expansion : pour le noeud courant v

Ordonner les actions en ordre croissant : **max** l'utilité, **min** la distance, **min** le nombre de noeuds visités

$$\text{score}_{\text{nbr}}(x) = -u(v, x \mid q) - 0.4 \cdot \frac{1}{1 + d(x, t)} + 0.2 \cdot \mathbf{1}[x \in V]$$

↑
distance la plus courte dans le sous-graphe

Éteindre $K(v)$ enfants

$$K(v) = \max \left(1, \left\lfloor 2 \cdot \sqrt{N(v)} \right\rfloor \right)$$

3.4

Simulation/rollout (profondeur maximale = 4)

Filtrer les noeuds non-visités $\mathcal{N}_{\text{rollout}}(v) = \{x \in \mathcal{N}(v) : x \notin V\}$

Choisir la meilleur action (epsilon-greedy)

$$a_t = \begin{cases} \text{best candidate} & \text{with prob. 0.9} \\ \text{random top-3 candidate in } \mathcal{N}_{\text{rollout}}(v) & \text{with prob. 0.1} \end{cases}$$

$$\text{terminal}(s) = \begin{cases} \text{target_reached} & v_t = \text{target} \\ \text{depth_limit} & |P_t| - 1 \geq 120 \\ \text{token_budget} & \tau_t \geq 12500 \\ \text{dead_end} & |\mathcal{A}(s)| = 0 \end{cases}$$

3.3 Expansion : pour le noeud courant v

1. Ordonner les actions : max l'utilité, min distance, min nœuds visités

$$\text{score}_{\text{nbr}}(x) = -u(v, x \mid q) - 0.4 \cdot \frac{1}{1 + d(x, t)} + 0.2 \cdot \mathbf{1}[x \in V]$$

distance la plus courte dans le sous-graphe

2. Expand au plus $K(v)$ enfants

$$K(v) = \max \left(1, \left\lfloor 2 \cdot \sqrt{N(v)} \right\rfloor \right)$$

3.4 Simulation/rollout (profondeur max = 4)

1. Filtrer les noeuds non-visités : $\mathcal{N}_{\text{rollout}}(v) = \{x \in \mathcal{N}(v) : x \notin V\}$

2. Choisir l'action (epsilon-greedy) :

$$a_t = \begin{cases} \text{best candidate} & \text{with prob. } 0.9 \\ \text{random top-3 candidate in } \mathcal{N}_{\text{rollout}}(v) & \text{with prob. } 0.1 \end{cases}$$

Conditions Terminales : terminal(s)

$$\text{terminal}(s) = \begin{cases} \text{target_reached} & v_t = \text{target} \\ \text{depth_limit} & |P_t| - 1 \geq 120 \\ \text{token_budget} & \tau_t \geq 12500 \\ \text{dead_end} & |\mathcal{A}(s)| = 0 \end{cases}$$

3.5

Backpropagation $P = (v_0, \dots, v_L)$

Évaluer la récompense : **max** l'utilité, bonus si le dernier état est l'état terminal, **pénaliser** la longueur du chemin et **pénaliser** le nombre de tokens

$$\text{mean_edge_utility}(P) = \frac{1}{L} \sum_{i=0}^{L-1} u(v_i, v_{i+1} \mid q) \qquad \text{token_ratio}(P) = \min \left(1, \frac{\sum_{e \in P} \text{tok}(e)}{T_{\max}} \right)$$

$$R(P) = \text{mean_edge_utility}(P) + \mathbf{1}[v_L = \text{target}] - 0.02 \cdot L - 0.15 \cdot \text{token_ratio}(P)$$

Pour chaque noeud sur le chemin s

$$N(s) \leftarrow N(s) + 1 \qquad \text{nombre de fois l'état } s \text{ a été visité}$$

$$W(s) \leftarrow W(s) + R(P) \qquad \text{la récompense cumulative}$$

$$\bar{Q}(s) = \frac{W(s)}{N(s)} \qquad \text{value function estimée de l'état } s$$

3.5 Backpropagation

Évaluer la récompense : 1. **Max** l'utilité. 2. **Bonus** si l'état est terminal. 3. **Pénaliser** la longueur. 4. **Pénaliser** le budget de tokens.

$$P = (v_0, \dots, v_L)$$
$$\text{mean_edge_utility}(P) = \frac{1}{L} \sum_{i=0}^{L-1} u(v_i, v_{i+1} \mid q)$$
$$\text{in_edge_utility}(\tilde{P}) + \mathbf{1}[v_L = \text{target}] - 0.02 \cdot L - 0.15 \cdot \text{tok} \quad \text{token_ratio}(P) = \min \left(1, \frac{\sum_{e \in P} \text{tok}(e)}{T_{\max}} \right)$$

Mise à jour pour chaque nœud sur le chemin **s**

$$N(s) \leftarrow N(s) + 1 \quad \text{nombre de fois l'état } s \text{ a été visité}$$

$$W(s) \leftarrow W(s) + R(P) \quad \text{la récompense cumulative}$$

$$\bar{Q}(s) = \frac{W(s)}{N(s)} \quad \text{value function estimée de l'état } s$$

Évaluation : Benchmarks

DATASET

2 Datasets de UltraDomain Benchmark : Mix & Agriculture

Embeddings: text-embedding-3-small

LLM (génération): OpenAI GPT 5.4 mini

LLM as Judge (LLMaJ): OpenAI GPT 5.4 mini

Dataset	Docs	Questions	Chunks	Nodes	Edges	Community Reports	Entity Vectors	Chunk Vectors
Agriculture	12	100	1,756	23,224	48,428	4,639	23,224	1,756
Mix	61	130	579	16,356	25,960	2,695	16,356	579

Évaluation : Performance LLMaJ

Agriculture : Analyse comparative des variantes sur un échantillon de 30 questions.

AGRICULTURE

Variant	Wins	Losses	Win Rate	Latency (s)	Tokens	Bridge Edges
hi_mcts	32	27	53.3%	2.04	4,990	14.9
hi_minmax	27	32	45.0%	5.14	5,141	17.6
hi_minmax_budgeted	32	28	53.3%	5.02	5,141	17.6
hi_rerank_weighted	26	34	43.3%	7.54	5,800	14.8
hi_weighted	33	26	55.0%	2.48	4,976	14.2
naive (Baseline)	53	7	88.3%	0.51	6,016	0.00

Évaluation : Performance LLMaJ (strict)

Agriculture : Analyse comparative des variantes sur un échantillon de 30 questions.

AGRICULTURE

Variant	Wins	Losses	Ties	Win Rate	Loss Rate	Tie rate	Latency (s)	Tokens	Bridge Edges
hi_mcts	32	27	1	53 %	0.45	0.02	2.04	4990	14.93
hi_minmax	27	32	1	45 %	0.53	0.02	5.14	5141	17.60
hi_min_budget	32	28	0	53 %	0.47	0.00	5.02	5141	17.60
hi_rerank	26	34	0	43 %	0.57	0.00	7.54	5800	14.80
hi_weighted	33	26	1	55 %	0.43	0.02	2.48	4976	14.20
naive (Baseline)	53	7	0	<u>88 %</u>	0.12	0.00	0.51	6016	0.00

Évaluation : Performance LLMaJ

Mix : Analyse comparative des variantes sur un échantillon de 30 questions.

MIX

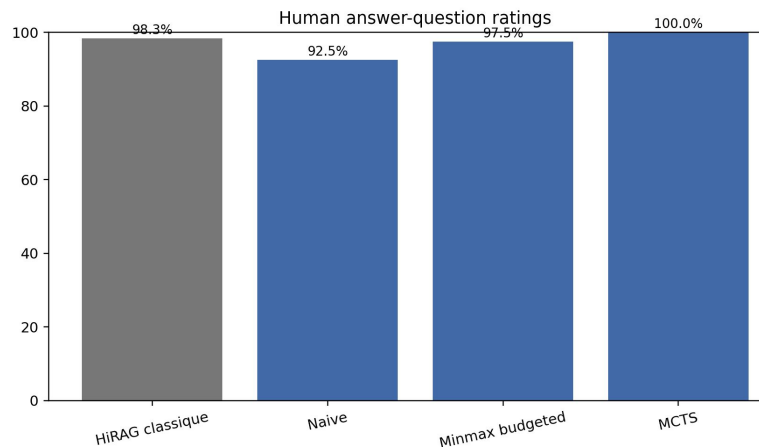
Variant	Wins	Losses	Ties	Win Rate	Loss Rate	Tie rate	Latency (s)	Tokens	Bridge Edges
hi_mcts	29	31	0	48 %	0.52	0.00	0.96	4573	12.40
hi_minmax	26	34	0	43 %	0.57	0.00	2.49	4576	12.33
hi_min_budget	23	37	0	38 %	0.62	0.00	2.43	4576	12.33
hi_rerank	25	35	0	42 %	0.58	0.00	5.01	4974	12.13
hi_weighted	24	36	0	40 %	0.60	0.00	1.19	4551	12.00
naive (Baseline)	40	20	0	<u>67 %</u>	0.33	0.00	0.41	6614	0.00



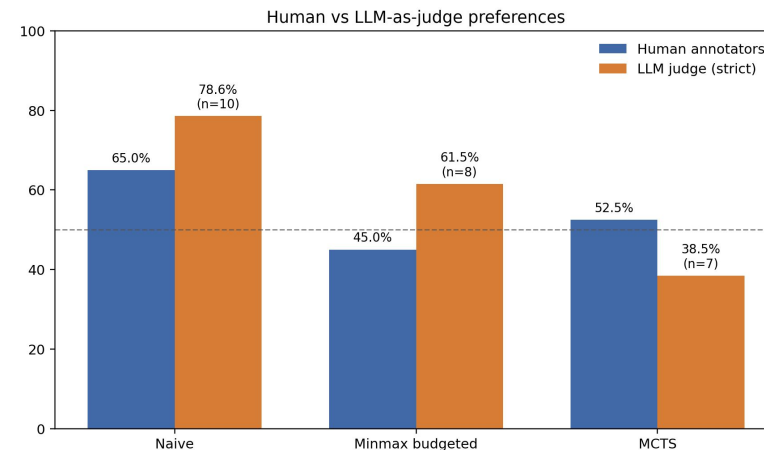
Évaluation : Performance et Préférences

Analyse comparative des scores de réponse et des préférences humain vs LLM-as-juge.

Scores de Réponse Humaine (%)



Préférences Humain vs LLM-as-juge

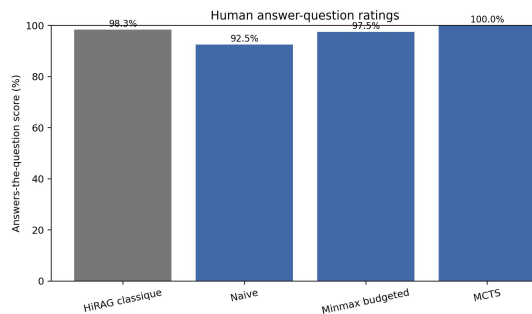




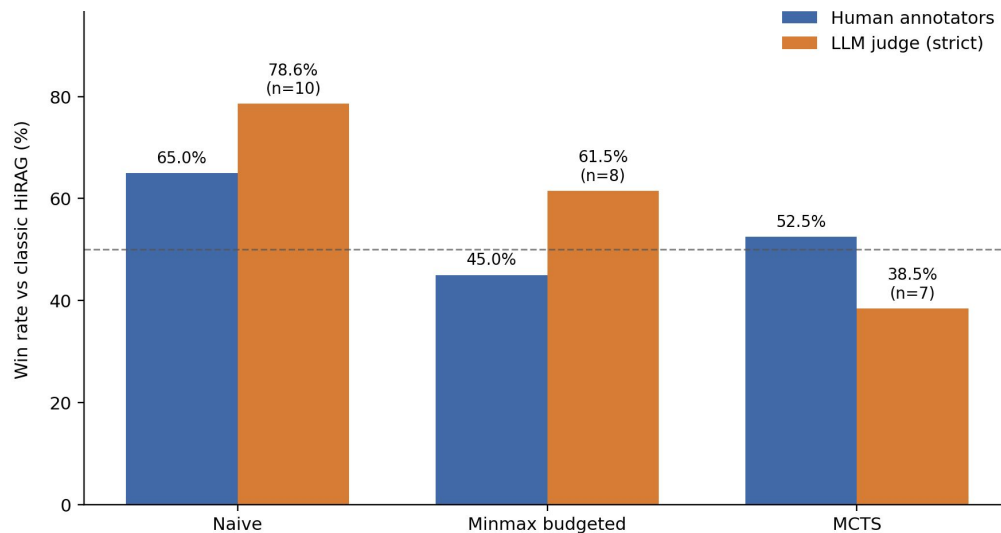
Évaluation : Performance et Préférences

Analyse comparative des scores de réponse et des préférences humain vs LLM-as-juge.

Scores de Réponse Humaine (%)



Préférences Humain vs LLM-as-juge



Évaluation : Performance LLMaJ

Mix : Analyse comparative des variantes sur un échantillon de 30 questions.

MIX

Variant	Wins	Losses	Win Rate	Latency (s)	Tokens	Bridge Edges
hi_mcts	29	31	48.3%	0.96	4,573	12.4
hi_minmax	26	34	43.3%	2.49	4,576	12.3
hi_minmax_budgeted	23	37	38.3%	2.43	4,576	12.3
hi_rerank_weighted	25	35	41.7%	5.01	4,974	12.1
hi_weighted	24	36	40.0%	1.19	4,551	12.0
naive (Baseline)	40	20	66.7%	0.41	6,614	0.00

Évaluation. Annotation humaine

But : comparer HiRAG classique à nos 3 variantes

Variante	Win rate	Densité span	Gain
HiRAG classique	-	28.7%	-
Naive	65.0%	32.0%	+0.30
Minmax budgeted	45.0%	32.1%	-0.10
MCTS	52.5%	30.4%	+0.05

Interface d'annotation

Packet 1 | Query mix:11

How did The Cleveland Show come into existence, and what was its premise?

Answer A

Development

The Cleveland Show came into existence as a spin-off of *Family Guy*. Seth MacFarlane initially conceived the series in 2007 while he was working on *Family Guy* and *American Dad!* for Fox. The show was developed around Cleveland Brown, a character already established on *Family Guy*, and Fox picked up the

Answer B

Creation and development

The Cleveland Show came into existence as a spin-off of *Family Guy*. According to the data, it was created by Seth MacFarlane, Richard Appel, and Mike Henry for Fox Broadcasting Company.

Your annotation

Does Answer A answer the question?

☒ yes ☐ partially ☐ no

Does Answer B answer the question?

☒ yes ☐ partially ☐ no

Which answer do you prefer?

☒ A ☐ B ☐ tie ☐ neither

Useful span from Answer A

The Cleveland Show came into existence as a spin-off of Family Guy. Seth MacFarlane initially conceived the series in 2007 while he was working on Family Guy and American Dad! for Fox. [...] The series centered on Cleveland Brown after he moved into a

Useful span from Answer B

Copy the smallest useful text span, or leave blank.

Press ⌘+Enter to apply

Optional notes

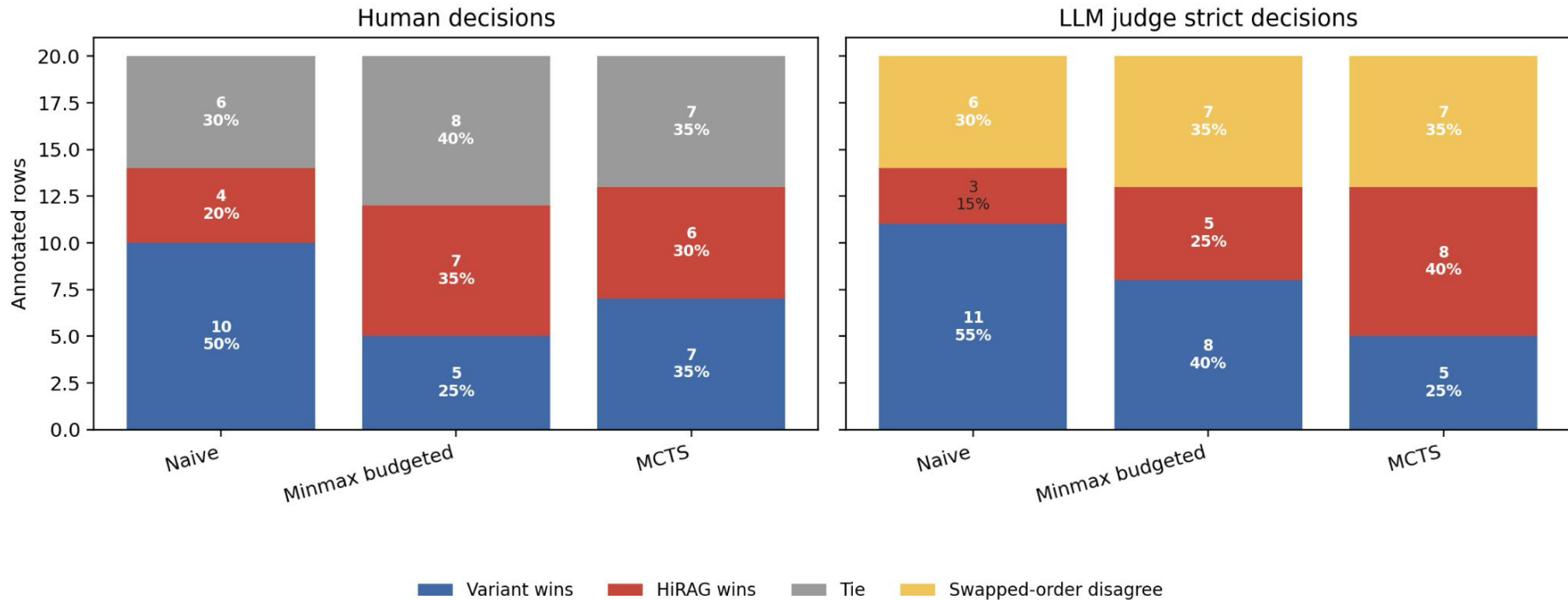
Uncertainty, factual concerns, or why neither answer is satisfactory.

Évaluation. Human annotation

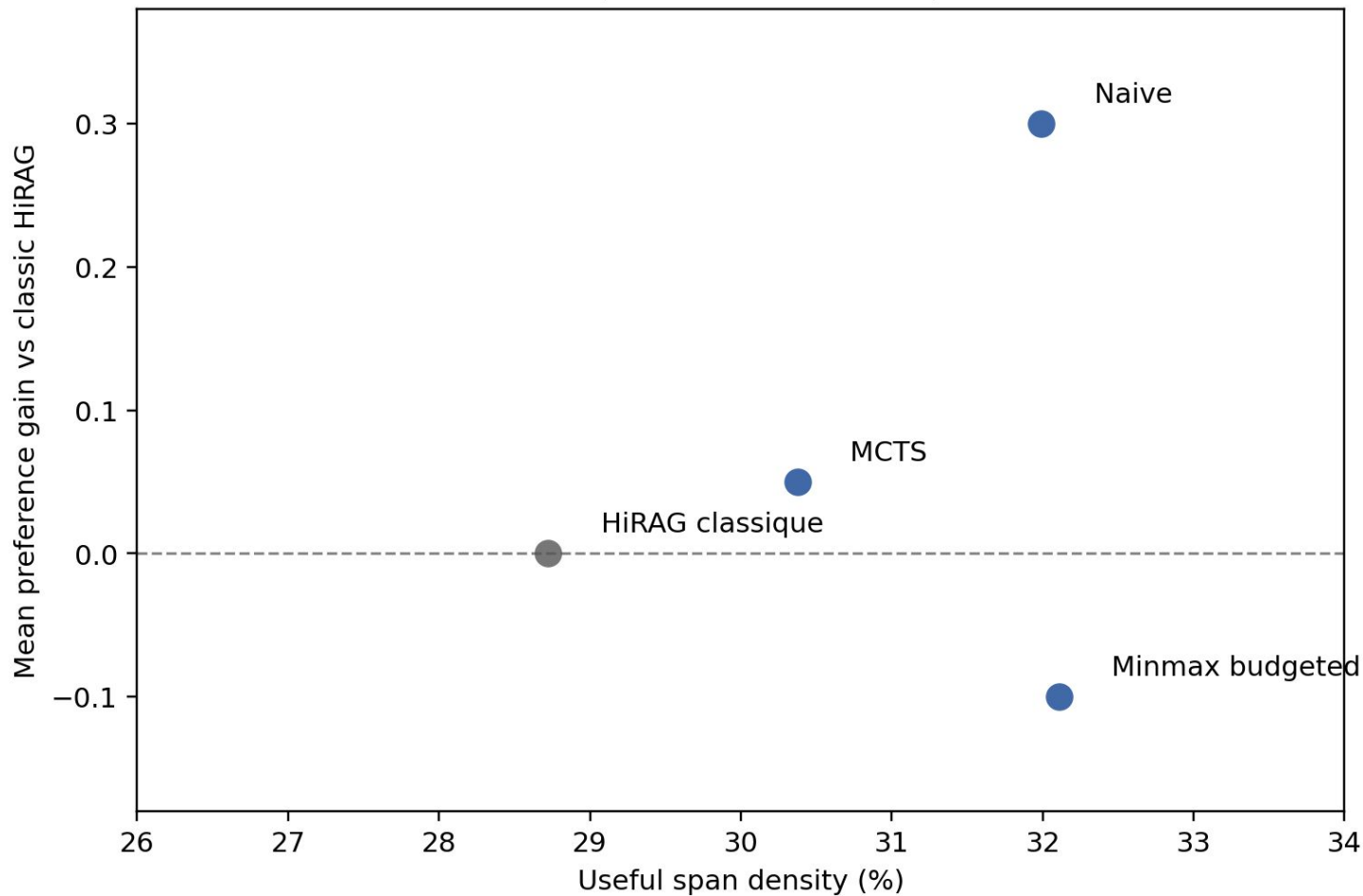
But : comparer HiRAG classique à nos 3 variantes (nouvelles données incluses) :

Variante	Human win rate vs HiRAG	LLM win rate vs HiRAG	Human gain	Span density
Naive	65.0%	78.6%	+0.30	32.0%
Minmax budgeted	45.0%	61.5%	-0.10	32.1%
MCTS	52.5%	38.5%	+0.05	30.4%

Decision breakdown on the same human-annotated pairs



Evidence compactness vs human preference



Conclusion

